

阅读文献笔记

SGLang 的 RadixAttention 与 vLLM 的 PagedAttention: 从 KV Cache 管理到在线推理服务

对应挑战：部署 SGLang、运行 Mooncake trace workload，并解释 RadixAttention、RadixCache、page-sized KV cache、prefix reuse 与 PagedAttention 的关系。

一、阅读范围与问题意识

本笔记阅读挑战材料列出的两篇核心论文：Zheng 等人在 NeurIPS 2024 发表的 SGLang 论文，以及 Kwon 等人在 SOSP 2023 发表的 vLLM PagedAttention

论文。两篇论文都把大模型推理看成一个受内存和调度约束的在线服务问题，但切入点不同：SGLang 更关注多次生成调用、共享前缀和结构化输出组成的 LM Program；PagedAttention 更集中地解决 KV Cache 的动态分配、碎片和共享问题。

这是我第一次比较系统地读推理系统论文。刚开始看到 RadixAttention、RadixCache、PagedAttention 和 KV Cache 这些词时，我感觉它们都在讲“缓存”，很容易混在一起。后来我强迫自己把问题拆成三层：模型计算层的 prefill/decode，KV Cache 的逻辑组织与物理存储，以及 scheduler 对请求执行顺序和批处理方式的控制。挑战中的流程图正好把这三层串起来，也成为我读完论文后检验自己是否真正理解的方式。

1.1 我第一次阅读这两篇论文的过程

我的第一遍没有直接从公式开始，而是先看摘要、Introduction 和系统架构图，先回答“作者到底想解决什么”。读 SGLang 时，我先被 LM Program 这个词卡住了：它并不只是普通 prompt，而是把多次调用、分支、并行和结构化输出写成程序。读到这里我才意识到，SGLang 优化的对象不是孤立的一次推理，而是一个有上下文关系的调用过程。

第二遍我按关键词回看方法部分，把每个概念写成自己的问题：RadixAttention 到底缓存什么？命中以后省掉的是哪段计算？page-sized KV cache 和 prefix reuse 是不是同一个东西？读到 vLLM 的 PagedAttention 后，我才把“逻辑上找到相同前缀”和“物理上把 KV 放进不连续显存”区分开。最后再看实验部分，我重点关注模型、GPU、请求分布和指标，因为论文里的加速数字不能脱离实验条件单独理解。

这个过程对我来说有一个明显转折：之前我会把“吞吐提升”理解成某个 kernel 更快，读完后才发现，很多收益来自少做了重复的 prefill、让显存容纳更多请求，以及让 scheduler 按更合适的顺序工作。也就是说，系统论文的关键不一定是一条特别复杂的公式，而是把多个环节一起设计。

1.2 小绿鲸与 Zotero 的使用方式

阅读工具上，我使用小绿鲸作为论文的主要阅读和标注环境：先导入两篇 PDF，通读摘要和引言，再回到方法、实验和图表部分做高亮。我在小绿鲸里把容易混淆的词放在同一处对照，例如把“prefix reuse”旁边标成“重复 prefill”，把“paged KV”旁边标成“物理显存布局”，这样第二次回看时不容易只记住名词。

Zotero 主要负责文献管理，而不是替我理解论文。我用它保存两篇论文的题名、作者、会议和 DOI，给文献加上“SGLang”“KV Cache”“LLM serving”等标签，并把阅读中形成的短笔记和挑战要求关联起来。这样做的好处是，写到参考文献时不需要重新从文件名猜论文信息，也能把 SGLang 论文和 PagedAttention 论文放在同一组里比较。

工具	我在第一次阅读中的用途	留下的结果
小绿鲸	打开 PDF、按摘要 - 方法 - 实验的顺序阅读；标注关键词、困惑点和图表结论	形成对 RadixCache、paged KV、scheduler 和实验指标的初步理解

工具	我在第一次阅读中的用途	留下的结果
Zotero	保存题录和 DOI，建立文献分组与标签，整理阅读笔记和引用信息	方便两篇论文交叉比较，也避免参考文献格式和来源遗漏

工具只能帮助我把材料放在一起，真正让我理解的是反复追问“这个设计改变了请求流程的哪一步”。这也是我在笔记里专门加入流程对应关系的原因。

论文	研究对象	核心问题	与本次挑战的关系
SGLang: Efficient Execution of Structured Language Model Programs Neur IPS 2024	LM Program 的前端语言与运行时	多次调用、分支/合并、共享前缀、结构化输出如何高效执行	解释 RadixAttention / RadixCache、prefix reuse 和 SGLang 请求流程
Efficient Memory Management for Large Language Model Serving with PagedAttention SOSP 2023	vLLM 的 KV Cache 管理与服务系统	动态 KV Cache 的碎片、过度预留以及跨序列共享	回答 PagedAttention 与 SGLang paged KV 机制的联系和区别

二、论文一：SGLang 的系统思想

2.1 为什么需要 LM Program 运行时

论文观察到，真实应用已经从一次 prompt、一次 completion，发展为包含多轮对话、工具调用、树状思考、few-shot、self-consistency、检索增强和多模态输入的程序。这样的程序具有两个共同特征：一是包含多个有依赖关系的 LLM 调用和控制流；二是输入输出往往需要结构化，便于继续组合。若只使用普通 OpenAI 风格接口，开发者必须手动拼接字符串、解析结果、实现并行和处理多模态模板，代码冗长且运行时看不到跨调用的优化机会。

SGLang 的架构由前端语言和后端运行时组成。前端以 Python 内嵌 DSL 提供 extend、gen、select、fork、join 等原语；解释器把 prompt state 作为异步流提交给运行时，取结果时才发生必要的同步。后端 SGLang Runtime 负责批处理、KV Cache 管理和解码优化。这个前后端协同很重要：前端知道程序的分支和共享关系，后端则能把这些信息转化为缓存命中和执行顺序。

2.2 RadixAttention / RadixCache 的核心机制

论文中的核心问题是：许多请求拥有相同的 token 前缀，而传统推理服务通常在请求结束后丢弃 KV Cache，下一次请求只能重新执行 prefix 的 prefill。SGLang 将已生成 prompt 和结果对应的 KV Cache 保留下来，以 token 序列为键组织成 radix tree。radix tree 的边可以携带一段 token 序列，因此比逐 token 的 trie 更紧凑；运行时可以进行最长前缀匹配、插入、分裂和淘汰。

- 命中路径：新请求到达后，运行时寻找最长的已缓存 token 前缀；命中部分直接复用其 KV Cache，只对未命中的后缀执行 prefill，再进入 decode。
- 物理存储：论文描述的 KV Cache 使用非连续的 paged layout，页面大小按一个 token 组织；缓存 token 与正在运行请求共享显存池。逻辑上的 radix tree 和物理上的 KV page 是两件互补的事。
- 容量管理：使用 LRU 优先淘汰最久未使用的叶节点；连续批处理时通过引用计数保护仍被运行请求使用的节点。等待请求很多时，可以牺牲缓存，把显存让给更大的运行 batch。
- 调度配合：论文提出 cache-aware scheduling，优先执行共享前缀较长的请求，以减少 cache thrashing；离线情况下，按 radix tree 深度优先访问可以达到最优缓存命中率。

因此，RadixAttention 不是一个只替换 attention 数学公式的 GPU kernel 名称，而是一个围绕 radix tree、KV Cache 生命周期、淘汰策略和请求调度设计的运行时机制。挑战中常把它简称为 RadixCache，是因为当前 SGLang 实现通常把“radix tree 中缓存的前缀 KV 状态”作为可查询、可复用的缓存来理解。

我第一次读到这里时，最容易产生的误解是把 radix tree 当成一种更快的 attention。后来结合图 3 的多轮对话和分支示例，我才看清它真正改变的是“请求到来后如何找到已经算过的前缀”，而不是改变 Transformer 的注意力定义。这个区别也帮助我理解了为什么它会和 scheduler、LRU 以及显存管理一起出现。

2.3 论文中另外两项优化

SGLang 论文还提出了压缩有限状态机和 API speculative execution。对于 JSON 等正则约束输出，普通约束解码每次只生成一个 token；压缩 FSM 把只有单一路径的连续状态合并，使多个确定 token 可以在一次 forward pass 中处理。对只提供 API 的模型，API speculative execution 则尝试让前一次调用多生成一些后续模板可能需要的 token，并在后续调用中复用匹配结果。这两项技术说明论文的范围不止是 KV Cache，但本次挑战的理论重点仍是 prefix reuse 和 paged KV。

三、论文二：vLLM 的 PagedAttention

3.1 PagedAttention 解决的内存问题

PagedAttention 论文从另一个瓶颈出发：在线服务必须同时处理很多请求，而每个请求的 KV Cache 会随着生成动态增长，最终长度通常未知。早期系统把一个请求的 KV Cache 放进连续内存，并按最大长度或预估长度预留空间，导致内部碎片（预留但没有使用的空间）和外部碎片（不同大小的连续区域难以拼接）。论文报告，在其对比的系统设置中，实际 token 状态只占预留 KV Cache 空间的一部分，这直接限制了 batch size 和吞吐。

PagedAttention 借鉴操作系统虚拟内存：把每个序列的 KV Cache 切成固定大小的 block，物理 block 不要求连续；逻辑 token 位置通过 block table 映射到物理显存。这样可以按需分配，减少内部和外部碎片，也可以让多个序列共享同一组只读 KV block。vLLM 再结合 iteration-level scheduling、抢占以及重计算或 CPU swap，构成完整服务系统。

3.2 共享、调度与代价

- 共享：并行采样和 beam search 的序列拥有相同 prompt，可以共享 prompt 对应的 KV blocks；写入新 token 时再通过复制或新 block 保证不同分支相互独立。
- 调度：PagedAttention 负责提高显存利用率，但它与 iteration-level scheduling 是互补的。调度让不同请求在 token iteration 级别交错，分页内存则让更多请求真正放得进显存。
- 代价：非连续 block 需要 block table 和额外寻址，论文的 attention kernel microbenchmark 显示单 kernel 存在额外开销；block 太小会增加访问和传输开销，太大又会增加内部碎片，因此 block size 需要结合 workload 选择。

论文实验在 A100 上使用 OPT-13B、OPT-66B、OPT-175B 和 LLaMA-13B，并合成 ShareGPT、Alpaca 等请求负载。其主要结论是，在相近延迟下，vLLM 相对当时系统可获得约 2-4 倍吞吐提升；在请求共享前缀、并行采样或 beam search 的场景中，块级共享带来的收益更明显。这个数字属于论文硬件、模型、基线和负载下的结果，不能直接当作本次 Qwen/Qwen3-0.6B 实验的预期数值。

读 PagedAttention 时，我反而更容易找到熟悉的类比：它像操作系统分页，把一整块连续空间拆成固定大小的页，需要时再映射到物理位置。不过这个类比也不能照搬，因为 LLM 的 KV Cache 会动态增长，生成还有前后依赖，系统还要在分页开销、共享收益和请求调度之间做取舍。把这个限制写下来后，我觉得自己才算从“记住类比”走到了“理解设计边界”。

四、按挑战要求回答：两套机制如何应对

4.1 RadixAttention / RadixCache 解决什么问题？

它解决的是多次请求之间重复计算相同 prompt prefix 的问题。对于一段 prefix，Transformer prefill 会计算每层的 key 和 value；这些 KV 只由 prefix token 决定，因此在后续请求仍以同一 prefix 开头时可以复用。RadixCache 用 radix tree 保存“token 序列 - KV Cache”的映射，配合最长前缀匹配、LRU 淘汰、引用计数和 cache-aware scheduling，让 reuse 从手工配置变成运行时自动行为。收益主要体现为减少重复 prefill、降低首 token 等待时间、节省显存带宽，并在多调用程序中提高整体吞吐。

4.2 page-sized KV cache 与 prefix reuse 位于 pipeline 哪一部分？

机制	所在环节	做什么	对请求流程的影响
Prefix reuse / RadixCache	scheduler/queue 与 prefill 之间的缓存匹配	根据 token 前缀查询 radix tree，得到已缓存 KV 的最长匹配段	命中段跳过重复 prefill，剩余后缀继续 prefill，然后进入 decode；未命中时退化为普通流程
Page-sized KV cache	prefill/decode 的 KV Cache memory manager	将 KV 按固定页面或 block 分配到非连续物理显存，并维护逻辑到物理的映射	prefill 产生的 KV 和 decode 每步新增的 KV 可以按需落入 page/block，避免按最大长度连续预留
Cache-aware scheduling	scheduler/queue	根据共享前缀长度和缓存状态安排等待请求	提高命中率，但可能造成请求重排和公平性问题

可以用一句话记忆：prefix reuse 决定“哪些 KV 可以直接复用”，page-sized KV cache 决定“这些 KV 在显存里如何灵活存放”。前者是逻辑匹配和计算跳过，后者是物理内存管理；两者组合起来才同时解决重复计算和显存碎片。

4.3 vLLM PagedAttention 与 SGLang 机制的联系和区别

比较维度	vLLM PagedAttention	SGLang RadixAttention / RadixCache
首要目标	减少动态 KV Cache 的内部/外部碎片，提高可容纳 batch size	自动发现和复用跨请求、跨调用的共享 token prefix
逻辑组织	每条序列由 block table 映射到物理 KV blocks	共享 prefix 由 radix tree 组织，支持匹配、分裂、插入和淘汰
物理布局	固定大小 block，物理位置不要求连续	论文中同样使用非连续 paged KV layout；论文描述页面按 token 组织，具体实现版本可能有不同 block 粒度
共享场景	并行采样、beam search、共享 prompt 等，重点是 block 级共享	多轮对话、few-shot、self-consistency、fork 分支和不同程序实例的共享前缀
调度关系	通常与 iteration-level scheduling、抢占和重计算/换入结合	使用最长共享前缀优先等 cache-aware scheduling，并与 continuous batching 结合
两者关系	偏向 KV 的物理内存虚拟化	偏向 prefix 的逻辑缓存和工作负载感知复用，可兼容 paged attention

因此不能把两者简单说成“一个是分页、一个是树，所以互不相关”。SGLang 论文明确指出 RadixAttention 可以与 paged attention 兼容：radix tree 管理逻辑共享关系，paged KV 管理物理存储。区别在于论文贡献的主要着力点不同：PagedAttention 首先把 KV Cache 从连续预留改造成按 block 分配；RadixAttention 首先把跨请求的 prefix reuse 系统化，并把缓存命中纳入调度。

五、与本次实验的对应关系

5.1 一次请求的可验证流程

挑战要求的流程可以具体解释为：Client 发送 OpenAI-compatible 请求；Tokenizer 把文本变成 token；scheduler 将请求放入队列并决定批次；运行时先做 prefix/cache 匹配，再对未命中部分执行 prefill；prefill 和后续 decode 产生或读取 KV Cache；sampling 选择下一个 token；服务端以 streaming output 返回 token，并记录 TTFT、总 latency、输入输出 token 数和 status。

在 workload 实验中，Mooncake trace 提供 input_length 和 output_length 等长度信息。脚本用 synthetic prompt 保持长度分布，再按 Poisson 到达过程安排请求。这里的实验意义不是复刻论文的绝对吞吐，而是把论文中的系统变量变成可观察记录：队列和到达过程影响等待，prefix 是否相同影响 cache hit，输入长度影响 prefill，输出长度影响 decode，页面/block 管理影响可并发请求数。

5.2 记录结果时应注意

- 明确版本和硬件：SGLang 0.5.14、Ray 2.56.0、Qwen/Qwen3-0.6B，以及 GPU/显存信息。
- 每条请求至少记录 request id、计划 input/output length、实际 input/output tokens、status、TTFT 或 latency；区分服务失败、超时和正常完成。
- 记录 Poisson 到达率和随机种子，说明 workload 是怎样生成的；不要只展示平均延迟。

- 不要把论文中的 6.4 倍或 2-4 倍直接写成自己的结果。当前实验应报告自己的观测，并说明模型规模、GPU 和请求负载与论文不完全相同。

六、第一次阅读后的理解与反思

6.1 对我来说最难的地方

这次最难的不是读懂某一句英文，而是把分散在不同章节里的概念重新拼成一条请求路径。论文分别讨论缓存、内存、调度和实验，我一开始读完一段就觉得“好像懂了”，但一旦要回答 page-sized KV cache 位于 pipeline 哪一步，就会发现自己只记住了名词。后来我用小绿鲸的标注把“逻辑匹配”和“物理分配”分开，再用 Zotero 的文献笔记把两篇论文放在一起对照，理解才慢慢稳定下来。

我现在对这两篇论文的直观理解是：SGLang 更像是在问“哪些请求可以共享过去已经做过的工作”，PagedAttention 更像是在问“这些不断增长的工作状态如何放进有限的显存”。前一个问题让我看到了 workload 结构，后一个问题让我看到了内存结构；两者合起来，才是在线推理系统的完整视角。

6.2 论文的贡献

SGLang 的价值在于把“语言模型程序的结构”引入推理运行时：fork 暴露分支共享，前端 hint 帮助运行时识别前缀，RadixAttention 再把这些关系落实为缓存和调度。PagedAttention 的价值在于把操作系统分页思想应用到动态增长的 KV Cache，并与服务调度共同设计。两者共同说明，LLM 推理的性能不只由模型 FLOPs 决定，内存生命周期、请求顺序和 workload 结构同样是核心问题。

6.3 局限与复现风险

- 两篇论文的实验都依赖特定硬件、模型、基线实现和请求分布；换成小模型、低并发或没有共享前缀的请求后，收益可能变小。
- RadixCache 的命中收益依赖 prefix 稳定性；若 prompt 中包含时间戳、随机字段或用户内容变化很大，树会变宽，LRU 淘汰也会更频繁。
- 缓存复用会占用显存。高并发时，如果等待请求的收益大于继续保留旧缓存，系统需要在 cache hit 和当前 batch size 之间做权衡。
- cache-aware scheduling 可能改变先来先服务的顺序，论文也承认公平性和 starvation 仍需与命中率一起考虑。
- paged KV 的 block/page 粒度不能随意设定：太小增加映射、kernel 和传输开销，太大增加碎片并降低共享机会。

6.4 可作为挑战结论的三条话

第一，RadixCache 让相同 prefix 的重复 prefill 变成可复用状态；第二，page-sized KV cache 让动态增长的 KV 不必占用一块最大长度的连续显存；第三，SGLang 与 vLLM 的设计可以组合理解：一个强调“识别和调度共享 prefix”，一个强调“高效地存放和共享 KV blocks”。这三点正好对应挑战要求的流程图和 workload 观测。

七、参考文献

- [1] Lianmin Zheng et al. SGLang: Efficient Execution of Structured Language Model Programs. NeurIPS 2024. https://papers.nips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html
- [2] Woosuk Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. Proceedings of SOSP 2023, pp. 611-626. DOI: 10.1145/3600006.3613165.
- [3] SGLang Documentation. <https://docs.sglang.ai/>
- [4] vLLM Documentation. <https://docs.vllm.ai/>

说明：本文中的论文结论来自上述两篇论文；“与本次实验的对应关系”是基于论文机制对挑战步骤的解释，不替代实际运行日志。